

РЕФЕРАТ

Расчетно-пояснительная записка содержит 53 страницы, 11 рисунков, 1 таблицу, 11 источников.

КОМПЬЮТЕРНАЯ ГРАФИКА, ТРЁХМЕРНОЕ МОДЕЛИРОВАНИЕ, ВИЗУАЛИЗАЦИЯ, Z-БУФЕР, SHADOW MAPPING, ОСВЕЩЕНИЕ, ПОИСК ПУТИ, PYTHON, NUMPY.

Объект исследования — процессы визуализации трёхмерных сцен и анимации движения объектов.

Цель работы — разработка программного обеспечения для трёхмерного моделирования движения автомобиля в городской среде с использованием алгоритмов машинной графики.

В процессе работы был проведен анализ методов удаления невидимых поверхностей, алгоритмов закраски и построения теней. Обоснован выбор алгоритма построчного сканирования с Z-буфером и метода теневых карт. Спроектирована и реализована архитектура приложения на языке Python. Разработан графический конвейер, включающий матричные преобразования, растеризацию, текстурирование и расчет освещенности по модели Ламберта. Реализован модуль навигации на основе волнового алгоритма для ориентированного графа.

В результате создано интерактивное приложение, обеспечивающее корректную визуализацию городской среды с динамическими тенями и анимацией движения автомобиля. Исследование показало, что механизм отсечения невидимой геометрии демонстрирует высокую практическую значимость для оптимизации рендеринга. Анализ производительности выявил квадратичную зависимость времени рендеринга от коэффициента масштабирования сцены.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ВВЕДЕНИЕ	7
1 Аналитическая часть	8
1.1 Формализация объектов синтезируемой сцены	8
1.2 Анализ алгоритмов удаления невидимых поверхностей	10
1.2.1 Алгоритм обратной трассировки лучей	10
1.2.2 Алгоритм Робертса	10
1.2.3 Алгоритм построчного сканирования с использованием буфера глубины. Алгоритм Z-буфера	12
1.2.4 Обоснование выбора	13
1.3 Анализ методов закрашивания	14
1.3.1 Плоское закрашивание	14
1.3.2 Закрашивание по методу Гуро	15
1.3.3 Закрашивание по методу Фонга	15
1.3.4 Обоснование выбора	16
1.4 Построение теней	16
1.4.1 Теневые объёмы	16
1.4.2 Метод карт теней	17
1.4.3 Обоснование выбора	18
1.5 Организация движения автомобиля и поиск пути	18
1.5.1 Алгоритм Дейкстры	18
1.5.2 Муравьиный алгоритм	18
1.5.3 Волновой алгоритм	19
1.5.4 Выбор алгоритма поиска пути	20
1.6 Текстурирование	20
1.7 Выводы по аналитической части	21
2 Конструкторская часть	22
2.1 Требования к программному обеспечению	22
2.2 Описание структур данных	22

2.3	Общий алгоритм построения изображения	24
2.4	Аффинные преобразования	25
2.5	Приведение к пространству камеры	26
2.6	Перспективная проекция	26
2.7	Алгоритм построчного сканирования с Z-буфером	27
2.8	Расчет освещения и теней	28
2.9	Организация поиска пути	29
2.10	Вывод	30
3	Технологическая часть	31
3.1	Выбор средств реализации	31
3.2	Структура программы	31
3.3	Реализация алгоритмов визуализации	33
3.4	Реализация алгоритма отрисовки сцены	36
3.5	Управление сценой и взаимодействие с пользователем	39
3.6	Описание интерфейса программы	40
3.7	Выводы по технологической части	43
4	Исследовательская часть	44
4.1	Цель и задачи исследования	44
4.2	Технические характеристики стенда	44
4.3	Методика проведения исследования	45
4.4	Результаты исследования	45
4.5	Анализ результатов	47
4.6	Выводы по исследовательской части	48
	ЗАКЛЮЧЕНИЕ	49
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	51
	ПРИЛОЖЕНИЕ А	53

ВВЕДЕНИЕ

Современные методы компьютерной графики позволяют создавать трёхмерные сцены, отражающие объекты и процессы реального мира. Одной из практических задач является визуализация движения транспортных средств в городской среде, где важно корректно отображать геометрию, освещение и взаимодействие с элементами дороги и препятствиями [1].

Разработка подобных систем требует продуманной организации сцены и применения алгоритмов, обеспечивающих удаление невидимых поверхностей, расчёт освещения, текстурирование, построение теней и реализацию движения объектов. При этом необходимо поддерживать достаточную скорость работы и удобство восприятия визуализации.

Целью курсовой работы является разработка программного обеспечения для трёхмерного моделирования движения автомобиля в городской среде.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) описать структуру трёхмерной сцены, включающую дороги, здания и элементы городской инфраструктуры;
- 2) выбрать и реализовать алгоритмы удаления невидимых поверхностей, закрашивания и построения теней;
- 3) обеспечить поддержку текстурирования и освещения от бесконечно удалённых источников света;
- 4) разработать систему управления виртуальной камерой;
- 5) реализовать поиск кратчайшего пути по дорожной сети и анимацию движения автомобиля от начальной до конечной точек;
- 6) объединить все компоненты в единую программную систему для визуализации движения.

Разрабатываемое приложение должно обеспечивать визуально достоверное отображение городской сцены с трёхмерными моделями зданий и дорог, поддержку текстур, реалистичное освещение и анимацию движения автомобиля по кратчайшему маршруту.

1 Аналитическая часть

В данной части проводится анализ существующих алгоритмов, математических моделей и программных технологий, применяемых для решения комплексной задачи трёхмерного моделирования движения автомобиля в условиях городской среды. Основной целью аналитической части является обоснованный выбор методов визуализации, удаления невидимых поверхностей, расчета освещенности, построения теней и алгоритмической организации движения объекта по кратчайшему маршруту.

1.1 Формализация объектов синтезируемой сцены

Трёхмерная сцена представляет собой модель участка городской территории, включающую в себя здания, дорожную сеть и прочие элементы окружающей среды. Все геометрические построения и расчеты производятся в единой декартовой системе координат. Для корректной визуализации и взаимодействия объектов необходимо определить состав элементов сцены и способ их математической формализации в виде структур данных.

- 1) **Дороги.** Дороги представляют собой совокупность полигонов, образующих плоскость, по которой осуществляется движение. Для целей логического моделирования движения автомобиля дорожная сеть дополнительно описывается двумя двумерными массивами: матрицей проходимости, определяющей статические препятствия и свободные участки, и матрицей разрешённых направлений, которая задает правила движения на каждом конкретном участке. Геометрически дорога описывается набором вершин и граней с наложенными текстурами, имитирующими дорожное покрытие.
- 2) **Здания.** Здания моделируются как набор простейших геометрических примитивов, таких как параллелепипеды и призмы различной конфигурации. Каждое здание описывается через полигональную сетку, состоящую из списков вершин в трехмерном пространстве и списков граней, соединяющих эти вершины. Для повышения визуальной реалистичности используется текстурирование фасадов изображениями окон, дверей и материалов стен.

3) **Малые архитектурные формы.** Малые архитектурные формы, включающие скамьи и фонарные столбы, моделируются как набор простейших геометрических примитивов. Каждый объект описывается через полигональную сетку, состоящую из списков вершин в трехмерном пространстве и списков граней, соединяющих эти вершины. Для повышения визуальной реалистичности используется текстурирование поверхностей.

4) **Автомобиль.** Автомобиль является ключевым динамическим объектом сцены. Его геометрическая модель формируется из сложного набора вершин, ребер и граней, образующих кузов и колеса. Положение автомобиля в пространстве однозначно задается вектором координат его геометрического центра и матрицей поворота, определяющей текущую ориентацию корпуса относительно осей координат. Модель поддерживает аффинные преобразования для имитации езды.

5) **Источник света.** Источник света описывается вектором направления световых лучей и интенсивностью излучения. В данной задаче используется модель бесконечно удаленного источника света, имитирующего солнечное освещение. Особенностью данной модели является допущение, при котором все световые лучи считаются параллельными друг другу, что упрощает расчеты теней и освещенности для всех объектов сцены.

6) **Камера.** Камера представляет собой виртуального наблюдателя, через которого пользователь воспринимает трехмерную сцену. Ее параметры включают положение в мировых координатах, точку направления взгляда и вектор ориентации верха камеры. Все объекты сцены визуализируются путем перевода их координат в систему отсчета камеры с последующей перспективной проекцией на плоскость экрана.

Для хранения геометрической информации об объектах сцены используется полигональное представление. При таком подходе каждая поверхность описывается множеством вершин с уникальными координатами и списком граней, которые ссылаются на индексы этих вершин. Данный формат хранения данных позволяет эффективно применять алгоритмы растеризации и трансформации [2].

1.2 Анализ алгоритмов удаления невидимых поверхностей

Одной из фундаментальных задач трёхмерной визуализации является устранение невидимых поверхностей. Суть задачи заключается в определении того, какие части объектов перекрываются другими объектами с точки зрения наблюдателя и, следовательно, не должны выводиться на экран монитора.

1.2.1 Алгоритм обратной трассировки лучей

Основная идея метода состоит в моделировании процесса распространения света в обратном направлении — не от источников, а от наблюдателя к объектам сцены [3]. Поскольку считается, что наблюдатель расположен на бесконечности вдоль положительного направления оси z , все испускаемые им лучи можно считать параллельными этой оси. Для каждого пикселя экрана формируется первичный луч, выпущенный из камеры.

Далее для данного луча определяется пересечение со всеми объектами. Из всех найденных точек выбирается та, которая имеет максимальную координату z , то есть находится ближе всего к наблюдателю.

Для расчетов освещения из точки пересечения проводятся вторичные лучи, направленные к каждому источнику света. Если на пути такого луча обнаруживается непрозрачный объект, точка считается находящейся в тени. В противном случае учитывается вклад освещения.

Преимущества: метод обеспечивает высокую степень реалистичности, так как опирается на физически корректное моделирование распространения света.

Недостатки: метод является вычислительно затратным, точное определение пересечений лучей с геометрическими объектами требует решения сложных математических уравнений, что существенно снижает скорость при программной реализации.

1.2.2 Алгоритм Робертса

Данный алгоритм работает в объектном пространстве и предназначен для обработки исключительно выпуклых тел, реализуется этот алгоритм в три этапа [2].

На первом этапе формируется исходная матрица V , содержащая информацию о каждой грани многогранника. Размерность матрицы составляет $4 \times n$, где n – количество граней. Каждый столбец соответствует коэффициентам уравнения плоскости вида $ax + by + cz + d = 0$, описывающей конкретную грань:

$$V = \begin{pmatrix} a_1 & a_2 & \dots & a_n \\ b_1 & b_2 & \dots & b_n \\ c_1 & c_2 & \dots & c_n \\ d_1 & d_2 & \dots & d_n \end{pmatrix} \quad (1)$$

Требуется обеспечить корректность формирования матрицы: любая точка внутри тела должна находиться с положительной стороны относительно каждой грани. Если для какой-либо грани это условие не выполняется, соответствующий столбец матрицы умножается на -1 . Для проверки используется внутренняя точка тела, координаты которой можно получить усреднением координат всех его вершин.

На втором этапе выполняется определение рёбер, экранируемых самим телом. Задаётся вектор направления взгляда $E = \{0, 0, -1, 0\}$. Для выявления невидимых граней вычисляется произведение:

$$R = E \cdot V$$

Компоненты результирующего вектора R , имеющие отрицательные значения, соответствуют невидимым граням.

Третий этап предполагает удаление рёбер, перекрываемых другими объектами сцены. Для проверки видимости конкретной точки на ребре строится луч, соединяющий позицию наблюдателя с анализируемой точкой. Точка является невидимой, если на пути луча встречается рассматриваемое тело как препятствие. Факт пересечения лучом тела определяется условием нахождения луча с положительной стороны относительно плоскости каждой грани данного тела.

Преимущества: обеспечение высокой точности результата, достигаемой за счёт работы в объектном пространстве, что позволяет выполнять вычисления непосредственно с математическим описанием тел.

Недостатки: высокая вычислительная трудоёмкость алгоритма, составляющая $O(n^2)$, где n — количество объектов в сцене. Также недостатком является требование к геометрии: все тела должны быть строго выпуклыми, что накладывает необходимость проведения дополнительных проверок корректности их описания и разбиения невыпуклых объектов на выпуклые части.

1.2.3 Алгоритм построчного сканирования с использованием буфера глубины. Алгоритм Z-буфера

Алгоритм Z-буфера относится к методам, работающим в пространстве изображения и служит для определения видимости фрагментов сцены. Его идея представляет собой расширение концепции буфера кадра за счёт добавления второго массива данных — буфера глубины, в котором для каждого пикселя хранится значение координаты z [2].

Основной принцип работы: перед началом рендеринга буфер кадра заполняется значениями фона, а буфер глубины инициализируется минимальными возможными значениями z . Далее сцена обрабатывается построчно. Для каждого активного многоугольника внутри текущей строки вычисляются границы его видимой области. Затем для каждого пикселя внутри этих границ интерполируется его значение глубины. Полученное значение сравнивается с соответствующей записью в буфере глубины: если новый фрагмент расположен ближе к наблюдателю, чем уже записанный, выполняется закраска пикселя и обновление буфера глубины. Если же глубина больше, пиксель игнорируется. Принцип работы алгоритма представлен на рисунке 1.1.

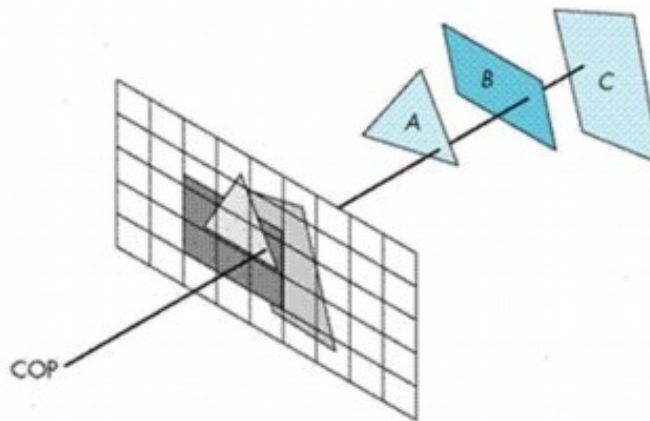


Рисунок 1.1 — Принцип работы Z-буфера: сохранение глубины ближайшего фрагмента

Преимущества: алгоритм не требует предварительной сортировки полигонов и позволяет выводить геометрические объекты в произвольном порядке. Благодаря этому Z-буфер эффективно обрабатывает сцены с большим количеством перекрывающихся объектов и корректно определяет ближайшие фрагменты. Сложность алгоритма масштабируется линейно с количеством обрабатываемых пикселей, что делает его удобным для применения в растровых графических системах.

Недостатки: метод требует выделения дополнительной памяти под буфер глубины, что может быть критично при высоких разрешениях. Также Z-буфер затрудняет корректную обработку прозрачных поверхностей, так как для них необходима сортировка фрагментов по глубине. Устранение ступенчатости контуров и реализация полупрозрачности требуют дополнительных процедур.

1.2.4 Обоснование выбора

Для реализации выбран алгоритм построчного сканирования с использованием буфера глубины. В отличие от алгоритма Робертса, данный подход корректно обрабатывает пересекающиеся объекты и не зависит от возможных ошибок в топологии моделей. По сравнению с методом трассировки лучей, выбранный алгоритм проще и эффективнее поддается векторизации на основе матричных операций, что позволяет обеспечить необходимую производитель-

ность.

1.3 Анализ методов закрашивания

Закрашивание определяет математический способ вычисления цвета каждой точки поверхности с учётом свойств материала и условий освещения. Выбор метода напрямую влияет на реалистичность и скорость рендеринга. Сравнение визуальных результатов различных методов представлено на рисунке 1.2.

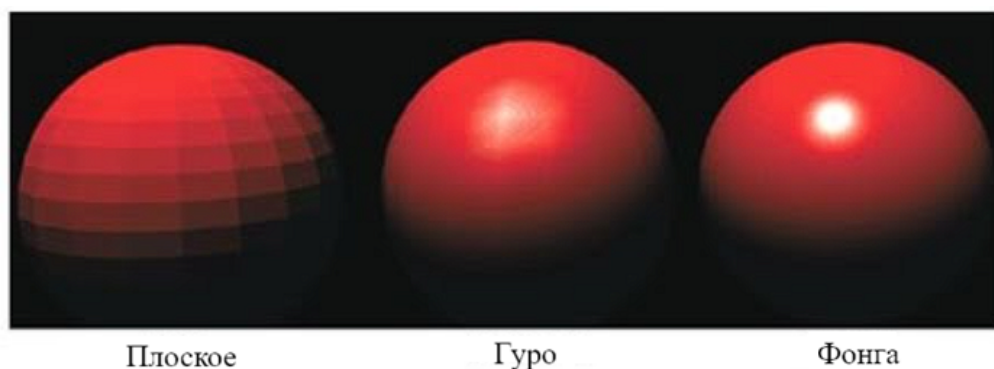


Рисунок 1.2 — Сравнение методов закрашивания: Плоское, Гуро, Фонга

1.3.1 Плоское закрашивание

При использовании данного метода для каждой грани вычисляется нормаль. Интенсивность освещённости рассчитывается по закону Ламберта как произведение интенсивности источника света на косинус угла падения лучей. Весь полигон целиком закрашивается одним цветом с рассчитанной интенсивностью, что придает объектам граненый вид. [2]

Преимущества: метод требует минимальных вычислительных затрат, так как расчет освещения выполняется один раз на целый полигон, а не для каждого пикселя в отдельности.

Недостатки: основным минусом является дискретность получаемого изображения. При аппроксимации криволинейных поверхностей отчетливо видны границы между смежными полигонами, из-за чего теряется гладкость формы. Этот визуальный дефект усиливается оптической иллюзией. Кроме того, метод не позволяет корректно отображать зеркальные блики: поскольку нормаль постоянна для всей грани, блик либо заливаает полигон целиком, либо отсутствует полностью.

1.3.2 Закрашивание по методу Гуро

Метод Гуро основан на линейной интерполяции интенсивности освещения по поверхности полигона. Процесс состоит из трех этапов: сначала вычисляются нормали в вершинах, затем для каждой вершины рассчитывается цвет согласно выбранной модели освещения, и, наконец, при растеризации цвет внутренних пикселей вычисляется путем билинейной интерполяции цветов вершин [2].

Преимущества: метод позволяет скрыть граненую структуру модели, создавая иллюзию гладкой поверхности. При этом он относительно вычислительно эффективен, так как сложные расчеты освещения (с векторами и косинусами) производятся только для вершин, количество которых значительно меньше количества пикселей.

Недостатки: основной проблемой метода является потеря зеркальных бликов, если они попадают в центр полигона, а не на вершину, то блик «размазывается» или исчезает при интерполяции.

1.3.3 Закрашивание по методу Фонга

Метод Фонга предполагает интерполяцию вектора нормали к поверхности для каждого пикселя. Вместо готового цвета, между вершинами интерполируются компоненты вектора нормали. Затем для каждого пикселя, используя полученную интерполированную нормаль, производится полный расчет освещенности по выбранной модели. [2]

Преимущества: обеспечивает наиболее реалистичное отображение криволинейных поверхностей среди локальных моделей освещения. Метод корректно воспроизводит четкие зеркальные блики в любой точке полигона и дает очень плавные градиенты освещенности.

Недостатки: крайне высокая вычислительная сложность. Необходимость нормализовать вектор и вычислять скалярные произведения для каждого пикселя экрана создает колоссальную нагрузку на процессор. При программной реализации без использования GPU этот метод часто приводит к неприемлемо низкому количеству кадров в секунду (FPS) для сцен с большим количеством объектов.

1.3.4 Обоснование выбора

Выбран метод плоского закрашивания. Поскольку рендеринг выполняется на центральном процессоре без использования графического ускорителя, применение более сложных методов приведет к значительному снижению частоты кадров. Простое закрашивание обеспечивает достаточную наглядность геометрии городских зданий, состоящих преимущественно из плоских поверхностей, при максимальной производительности системы.

1.4 Построение теней

Тени являются важным элементом восприятия глубины, позволяющим зрителю определить взаимное расположение объектов в пространстве.

1.4.1 Теневые объёмы

Теневые объёмы (Shadow Volumes) — это геометрический метод построения теней, который создаёт виртуальные объёмы за объектами, внутри которых точки сцены находятся в тени. Идея метода заключается в том, чтобы определить силуэтные грани объектов относительно источника света и вытянуть их вдаль, формируя объём, через который можно проверить, какие пиксели будут затенены. Для этого обычно используется буфер трафарета, позволяющий классифицировать каждый пиксель как освещённый или находящийся в тени [4].

Преимущества: метод обеспечивает высокую точность построения теней с чёткими границами, не зависит от масштаба или расстояния до камеры, а также не требует специальных параметров для устранения артефактов самозатенения. Использование видеокарты позволяет значительно ускорить обработку и сделать метод применимым к сложным сценам.

Недостатки: высокая вычислительная нагрузка, особенно при сложной геометрии, требует значительных ресурсов процессора. В реальных проектах часть вычислений обычно переносится на GPU, что делает метод более эффективным, но сохраняет высокие требования к аппаратуре. Также с помощью этого метода сложно реализовать тени для полупрозрачных объектов.

1.4.2 Метод карт теней

Алгоритм построения теней с использованием Z-буфера является двухпроходным [5]. На первом этапе сцена визуализируется с позиции источника света, при этом сохраняется только информация о глубине поверхностей в специальный буфер. На втором этапе сцена рендерится с точки зрения наблюдателя, а координаты каждой видимой точки преобразуются в пространство света и проверяются на видимость относительно источника. Если точка оказывается невидимой для света, она затемняется, формируя соответствующую тень. Такая реализация позволяет корректно отображать тени на кривых поверхностях и учитывать самозатенение объектов, при этом все дополнительные вычисления ведутся в пространстве изображения, что делает алгоритм относительно универсальным и независимым от сложности сцены. Пример обработки сцены представлен на рисунке 1.3

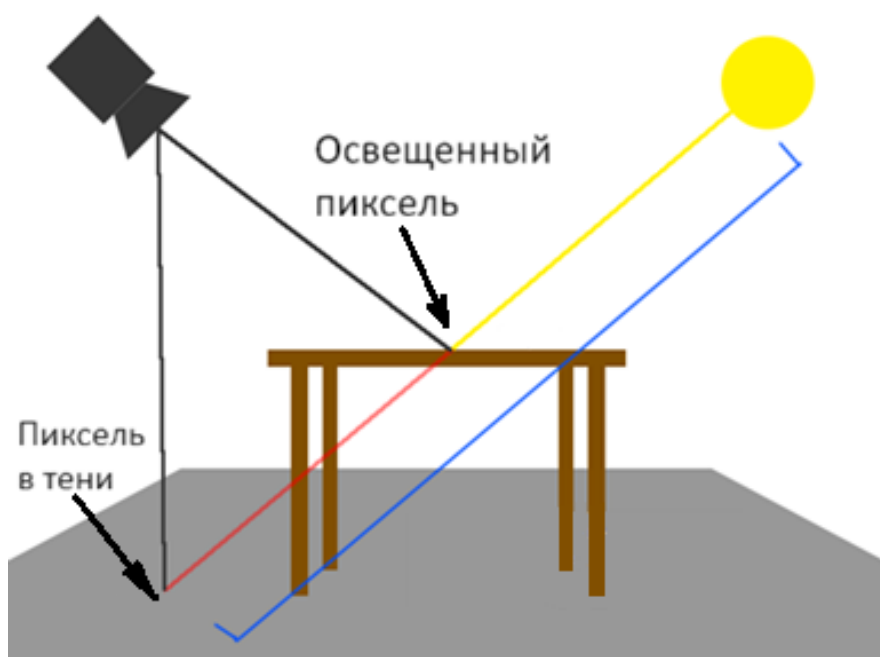


Рисунок 1.3 — Двухпроходный алгоритм построения карт теней

Преимущества: универсальность метода позволяет теням корректно ложиться на любые объекты независимо от их формы, а также автоматически поддерживает самозатенение [6].

Недостатки: качество теней напрямую зависит от разрешения карты глубины: при его недостатке края теней становятся пикселизированными. Из-

за дискретности буфера часто возникают артефакты, для устранения которых требуется подбор параметра смещения, что может привести к визуальному отрыву тени от объекта. Кроме того, необходимость двукратной отрисовки геометрии существенно увеличивает вычислительную нагрузку.

1.4.3 Обоснование выбора

Выбран метод карт теней. Проекционные тени недостаточно реалистичны для городской среды, где тени могут падать на стены зданий и другие машины. Метод карт теней идеально интегрируется в выбранный конвейер с использованием буфера глубины, позволяя использовать схожие алгоритмы растеризации для обоих проходов визуализации.

1.5 Организация движения автомобиля и поиск пути

Для реализации автоматического движения автомобиля необходимо выбрать алгоритм поиска кратчайшего пути на графе дорожной сети.

1.5.1 Алгоритм Дейкстры

Алгоритм Дейкстры предназначен для поиска кратчайших путей от одной вершины графа до всех остальных. Он работает с графами, имеющими неотрицательные веса ребер. Алгоритм последовательно выбирает вершину с наименьшей меткой расстояния и релаксирует ребра, исходящие из неё [7].

Преимущества: гарантированно находит кратчайший путь.

Недостатки: в случае равномерной сетки, где веса всех ребер равны единице, алгоритм Дейкстры выполняет избыточные операции с очередью с приоритетом, что делает его менее эффективным по сравнению с более простыми методами обхода.

1.5.2 Муравьиный алгоритм

Муравьиный алгоритм является метаэвристическим методом оптимизации, вдохновлённым поведением настоящих муравьёв при поиске кратчайших путей к источнику пищи. Алгоритм использует виртуальных «муравьёв», которые перемещаются по графу, откладывая и следуя феромонным следам. С течением времени наиболее успешные пути накапливают больше феромонов,

что повышает вероятность их выбора последующими муравьями [8].

Преимущества: способность находить хорошие решения для сложных комбинаторных задач, устойчивость к локальным минимумам, возможность параллельного выполнения.

Недостатки: относительно низкая скорость сходимости для больших графов, необходимость подбора параметров (скорость испарения феромонов, влияние феромонов и эвристики), результаты могут быть стохастически изменчивыми и требовать многократного запуска для получения стабильного решения.

1.5.3 Волновой алгоритм

Волновой алгоритм является вариацией поиска в ширину на графе. Он работает путем распространения волны от стартовой точки во все стороны равномерно, рисунок 1.4. Поскольку дорожная сеть представлена в виде сетки с одинаковыми весами ребер, волна гарантированно находит кратчайший путь [9].

9	10		10	9	8	9	10	11	12	13	14
8	9		9	8	7	8	9	10	11	12	13
7	8	9	8	7	6	7	8	9	10	11	12
6	7	8	7	6	5	6	7			10	11
5					4	5	6	7	8	9	10
4	3	2	1	2	3	4	5	6			11
3	2	1	0	1	2	3	4	5			10
4	3	2	1	2	3	4	5	6	7	8	9

Рисунок 1.4 — Распространение волны в алгоритме поиска пути на сетке

Преимущества: идеально подходит для дискретных сеток и лабиринтов. Легко модифицируется для учета запрещенных направлений движения путем проверки матрицы смежности или матрицы направлений.

Недостатки: Алгоритм является «слепым», то есть он исследует пространство равномерно во все стороны, не учитывая направление на целевую точку. Это приводит к посещению большого количества лишних вершин и снижению производительности на картах большого размера по сравнению с

эвристическими алгоритмами.

1.5.4 Выбор алгоритма поиска пути

Для реализации выбран модифицированный волновой алгоритм для ориентированного графа.

Обоснование: дорожная сеть в проекте представлена в виде клеточной матрицы, где расстояние между соседними клетками всегда равно единице. В таких условиях волновой алгоритм является оптимальным решением, гарантирующим нахождение кратчайшего пути. Кроме того, использование дополнительной матрицы направлений позволяет легко интегрировать в алгоритм правила дорожного движения, запрещая распространение волны против разрешенного потока.

1.6 Текстурирование

Для повышения визуальной реалистичности сцены без увеличения сложности геометрии используется метод UV-развертки текстур. Этот метод позволяет отображать двумерное изображение (текстуру) на трёхмерной поверхности модели. Каждой вершине треугольника присваиваются координаты (u, v) , которые указывают на соответствующую точку на текстуре. В процессе растеризации треугольника для каждого пикселя вычисляются интерполированные значения этих координат, что позволяет корректно отображать текстуру на поверхности с учётом её формы и ориентации [10].

UV-развертка особенно полезна для сложных моделей, где прямое наложение текстуры было бы невозможным или привело бы к значительным искажениям. Она обеспечивает точное соответствие текстурных деталей геометрии объекта и позволяет создавать реалистичные материалы без увеличения количества полигонов.

Метод UV-развертки обеспечивает оптимальный баланс между качеством визуализации и производительностью. Он позволяет достигать высокой скорости рендеринга и чёткости текстур, не нагружая центральный процессор, что особенно важно для рендеринга при ограниченных ресурсах. Основные ограничения метода включают возможную пикселизацию при увеличении масштаба текстуры и необходимость аккуратного построения UV-развертки для

минимизации растяжений и швов на поверхности модели.

1.7 Выводы по аналитической части

В ходе всестороннего анализа предметной области и существующих алгоритмов компьютерной графики был сформирован стек методов.

- 1) **Визуализация:** выбран алгоритм с использованием Z-буфера, так как он обеспечивает корректное отображение геометрии и устойчив к ошибкам моделирования.
- 2) **Тени:** выбран метод карт теней, который обеспечивает построение реалистичных динамических теней от объектов любой формы.
- 3) **Освещение:** выбрано плоское закрашивание по модели Ламберта, предоставляющее оптимальный баланс между качеством визуализации объёма и скоростью работы на центральном процессоре.
- 4) **Текстурирование:** выбран метод с использованием UV-развертки и выборкой ближайшего соседа, что позволяет повысить детализацию сцены при минимальных вычислительных затратах.
- 5) **Логика движения:** выбран волновой алгоритм на ориентированном графе, позволяющий строить корректные маршруты с учётом правил дорожного движения и односторонних дорог.

Сформированный набор методов и алгоритмов позволяет создать систему моделирования движения автомобиля.

2 Конструкторская часть

В данной части представлены требования к разрабатываемому программному обеспечению для трехмерного моделирования движения автомобиля, а также детально описаны алгоритмы и структуры данных, выбранные для реализации системы визуализации.

2.1 Требования к программному обеспечению

Разрабатываемая программа должна предоставлять пользователю графический интерфейс со следующим функционалом:

- загрузка и отображение трехмерной сцены городской среды из конфигурационных файлов;
- управление положением и ориентацией виртуальной камеры в пространстве;
- запуск анимации движения автомобиля по автоматически построенному маршруту;
- отображение текущих параметров производительности (кадров в секунду).

Разработанное ПО должно соответствовать следующим техническим требованиям:

- использование алгоритма Z-буфера для корректного удаления невидимых поверхностей;
- реализация метода карт теней (Shadow Mapping) для построения динамических теней;
- применение модели освещения Ламберта с плоским закрашиванием (Flat Shading);
- поддержка текстурирования объектов сцены;
- расчет маршрута движения с учетом графа дорожной сети и правил дорожного движения.

2.2 Описание структур данных

Для реализации алгоритмов визуализации и моделирования движения используются следующие ключевые структуры данных.

- 1) **Сцена (Scene)** — контейнер верхнего уровня, содержащий списки всех статических (здания, дороги) и динамических (автомобиль) объектов, параметры источника света и объект камеры.
- 2) **Геометрический объект (SceneObject)** включает в себя:
 - массив вершин (dots) — массив точек в трехмерном пространстве (x, y, z) ;
 - массив граней (faces) — массив полигонов, где каждый полигон задается индексами вершин, цветом и текстурными координатами;
 - вектор положения центра объекта в мировых координатах;
 - матрицу поворота, определяющую ориентацию объекта;
 - вектор направления «вперед» (forward) для корректного движения.
- 3) **Камера (Camera)** содержит:
 - вектор положения в мировом пространстве;
 - вектор направления взгляда (target);
 - вектор ориентации «вверх» (up);
 - матрицы вида (view_matrix) и проекции, пересчитываемые при изменении параметров.
- 4) **Источник света (ShadowCaster)** включает:
 - вектор направления света (для направленного источника);
 - параметры интенсивности и цвета;
 - собственные матрицы вида и проекции для генерации карты теней;
 - буфер глубины (Shadow Map) — двумерный массив для хранения расстояний от источника света до поверхностей.
- 5) **Дорожная сеть** представлена двумя матрицами:
 - матрица проходимости (бинарная: 0 — препятствие, 1 — дорога);
 - матрица направлений (битовая маска разрешенных направлений движения из каждой клетки).

2.3 Общий алгоритм построения изображения

Процесс генерации кадра состоит из двух основных проходов: прохода для генерации карты теней и прохода для финальной отрисовки сцены с точки зрения камеры. Графическая схема общего алгоритма представлена на рисунке 2.1.

На вход алгоритма подаются списки объектов сцены и параметры камеры. На выходе формируется двумерное растровое изображение, которое выводится на экран.

Этап 1: Генерация карты теней

- 1) Устанавливается камера в положение источника света.
- 2) Используется ортогональная проекция для имитации солнечных лучей.
- 3) Производится растеризация всех объектов сцены, но вместо цвета в буфер записывается только глубина (z -координата) ближайшего фрагмента.

Этап 2: Финальная отрисовка

- 1) Устанавливается камера наблюдателя.
- 2) Инициализируются буфер цвета и Z-буфер (заполняется значением бесконечности).
- 3) Для каждого объекта сцены выполняется конвейер визуализации:
 - преобразование вершин — переход в экранные координаты;;
 - отсечение невидимых полигонов;
 - растеризация полигонов методом построчного сканирования;
 - для каждого пикселя выполняется тест глубины, расчет освещения и наложение текстур.

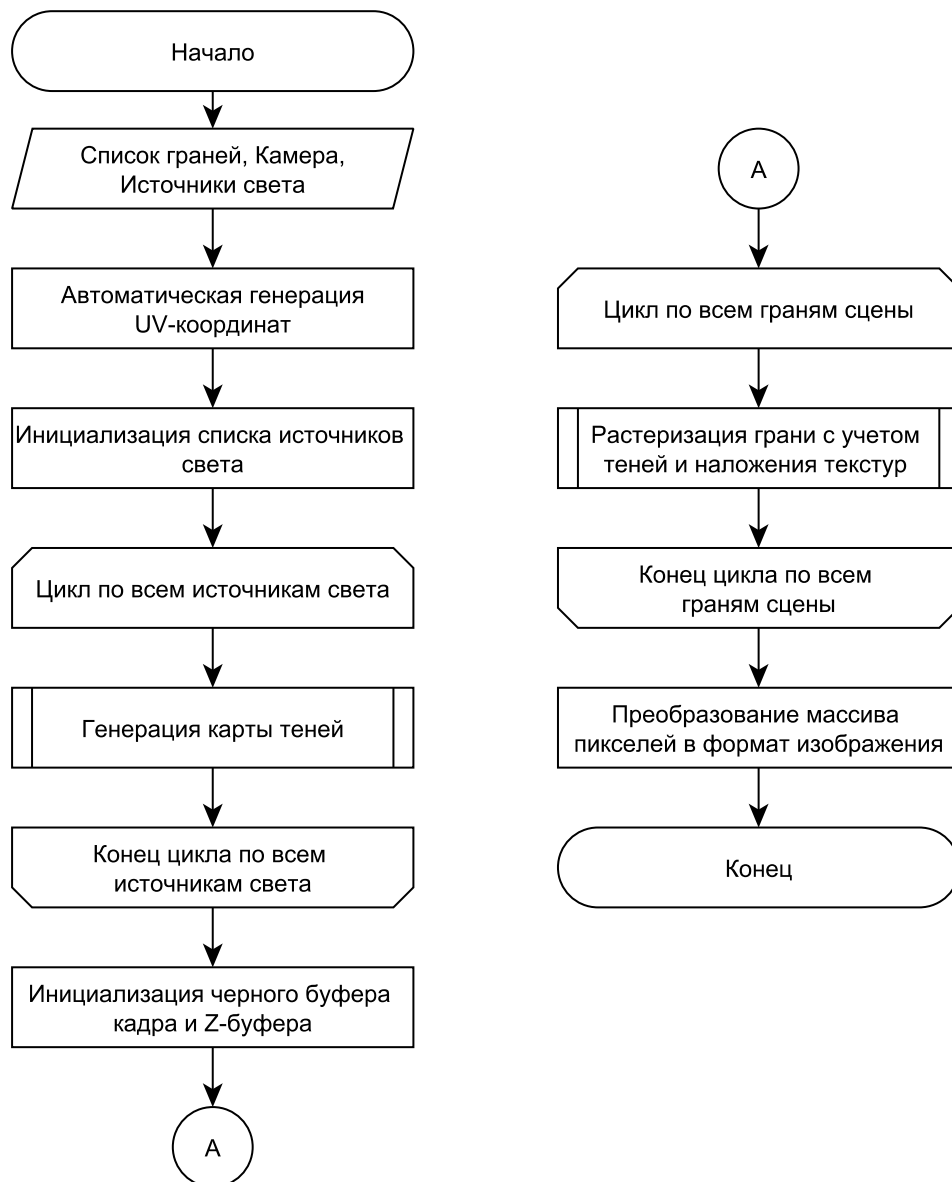


Рисунок 2.1 — Схема общего алгоритма генерации кадра

2.4 Аффинные преобразования

Для размещения объектов в сцене и анимации движения используются матрицы аффинных преобразований размером 4×4 в системе однородных координат.

Матрица поворота вокруг оси Z на угол α (используется для поворота

автомобиля):

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.1)$$

Матрица переноса на вектор (dx, dy, dz) :

$$T(dx, dy, dz) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ dx & dy & dz & 1 \end{pmatrix} \quad (2.2)$$

Итоговое положение вершины V_{new} вычисляется как произведение исходной вершины V на произведение матриц трансформации.

2.5 Приведение к пространству камеры

Для отображения сцены с точки зрения наблюдателя все объекты переводятся в систему координат камеры. Матрица вида (*ViewMatrix*) формируется на основе позиции камеры P , точки направления взгляда T и вектора «вверх» U .

Базис камеры рассчитывается следующим образом:

$$D = \frac{T - P}{\|T - P\|}, \quad R = \frac{D \times U}{\|D \times U\|}, \quad U' = R \times D \quad (2.3)$$

где D — вектор направления, R — вектор «вправо», U' — скорректированный вектор «вверх».

Матрица вида представляет собой комбинацию поворота и переноса, переводящую мировые координаты в систему отсчета, где камера находится в начале координат и направлена вдоль оси Z .

2.6 Перспективная проекция

Для создания эффекта перспективы используется матрица перспективной проекции. Она преобразует усеченную пирамиду видимости в куб в нор-

мализованных координатах устройства.

Ключевым этапом является **перспективное деление**: координаты x и y делятся на глубину z .

$$x_{screen} = x \cdot \frac{d}{z + d} \cdot scale + x_{offset} \quad (2.4)$$

$$y_{screen} = y \cdot \frac{d}{z + d} \cdot scale + y_{offset} \quad (2.5)$$

где d — фокусное расстояние, определяющее степень перспективного искажения.

2.7 Алгоритм построчного сканирования с Z-буфером

Для растеризации полигонов (преобразования векторных треугольников в пиксели) используется алгоритм построчного сканирования, совмещенный с Z-буфером для удаления невидимых поверхностей. Детальная схема алгоритма представлена на рисунке 2.2.

Схема работы алгоритма:

- 1) вершины треугольника проецируются на экранную плоскость;
- 2) определяется ограничивающий прямоугольник полигона по оси Y ;
- 3) происходит итерация по строкам экрана (y) от минимальной до максимальной координаты Y полигона;
- 4) для каждой строки находятся точки пересечения сканирующей линии с ребрами треугольника (x_{start}, x_{end});
- 5) внутри интервала $[x_{start}, x_{end}]$ для каждого пикселя интерполируется глубина z ;
- 6) **Z-тест**: значение z сравнивается с текущим значением в буфере глубины $z_{buffer}[y][x]$
 - если $z < z_{buffer}[y][x]$, то пиксель считается видимым. В буфер записывается новое значение z , а в буфер кадра — вычисленный цвет пикселя;
 - иначе пиксель отбрасывается.

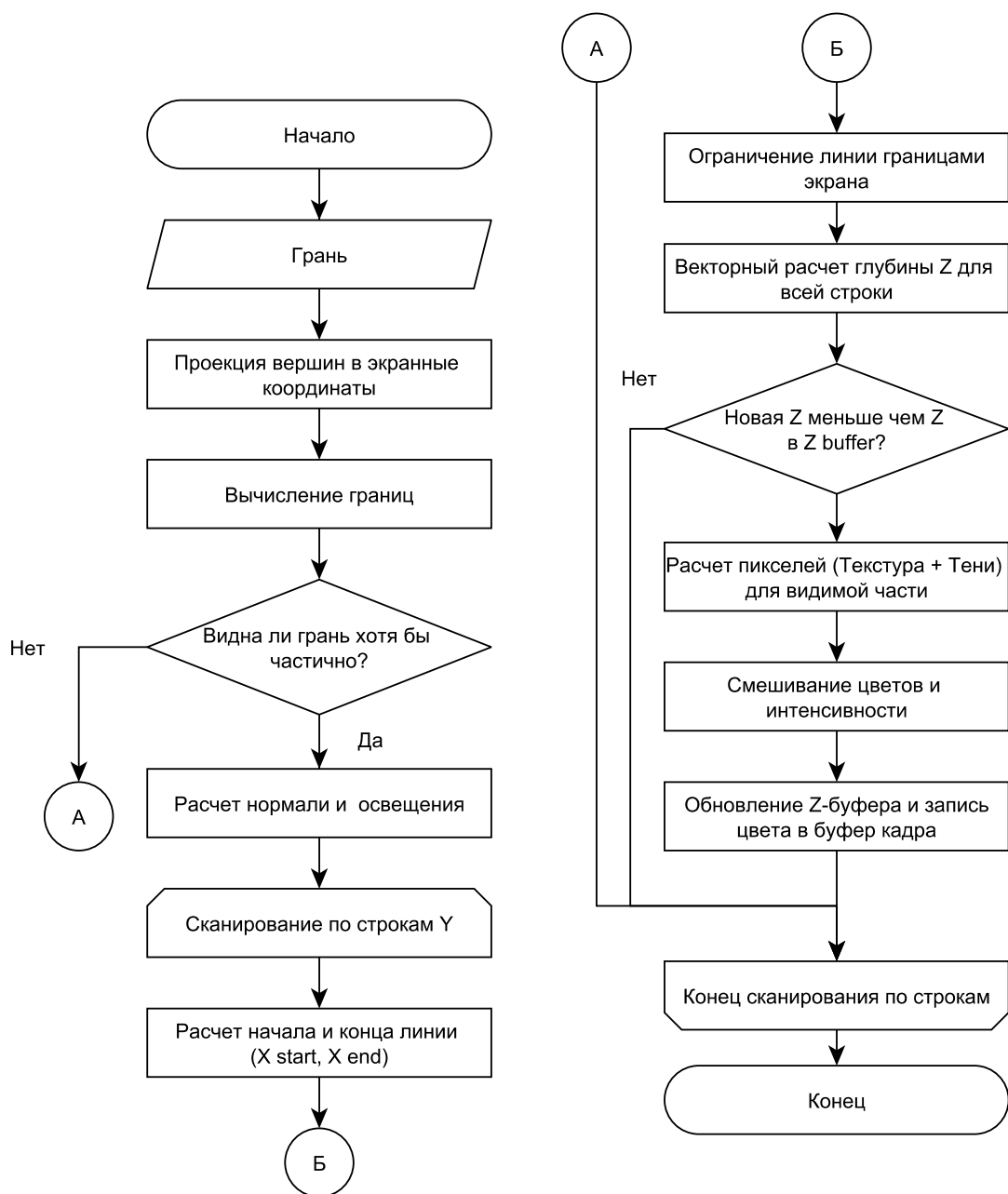


Рисунок 2.2 — Схема алгоритма построчного сканирования с Z-буфером

2.8 Расчет освещения и теней

Для закраски пикселей используется модель плоского закрашивания с учетом освещенности по закону Ламберта и теней.

- 1) **Нормаль поверхности:** для каждого треугольника вычисляется вектор нормали N как векторное произведение двух его сторон.
- 2) **Диффузное освещение:** коэффициент освещенности $I_{diffuse}$ рассчитывается как скалярное произведение нормали N и вектора направления

на свет L :

$$I_{diffuse} = |N \cdot L| \quad (2.6)$$

3) Расчет тени:

- координаты текущего пикселя (x, y, z) переводятся в пространство источника света;
- полученные координаты (u_{light}, v_{light}) используются для выборки глубины из карты теней: $d_{shadow} = \text{ShadowMap}[v_{light}, u_{light}]$;
- текущая глубина пикселя в пространстве света z_{light} сравнивается с d_{shadow} ;
- если $z_{light} > d_{shadow} + \text{bias}$, то точка находится в тени. Коэффициент освещенности уменьшается до значения фонового света.

Итоговый цвет пикселя вычисляется как произведение цвета текстуры на результирующий коэффициент освещенности.

2.9 Организация поиска пути

Для моделирования движения автомобиля используется **волновой алгоритм** на ориентированном графе. Схема разработанного алгоритма приведена на рисунке 2.3.

- 1) Дорожная карта дискретизируется в сетку.
- 2) Каждой ячейке сопоставляется битовая маска разрешенных направлений (Вверх, Вниз, Влево, Вправо).
- 3) Алгоритм начинает распространение волны от стартовой точки. При переходе в соседнюю клетку проверяется битовая маска. Это гарантирует соблюдение правил дорожного движения (например, запрет движения против потока).
- 4) После достижения целевой точки путь восстанавливается обратным ходом от финиша к старту.

Найденный путь преобразуется в список мировых координат, между которыми происходит плавная интерполяция положения и угла поворота автомобиля при анимации.

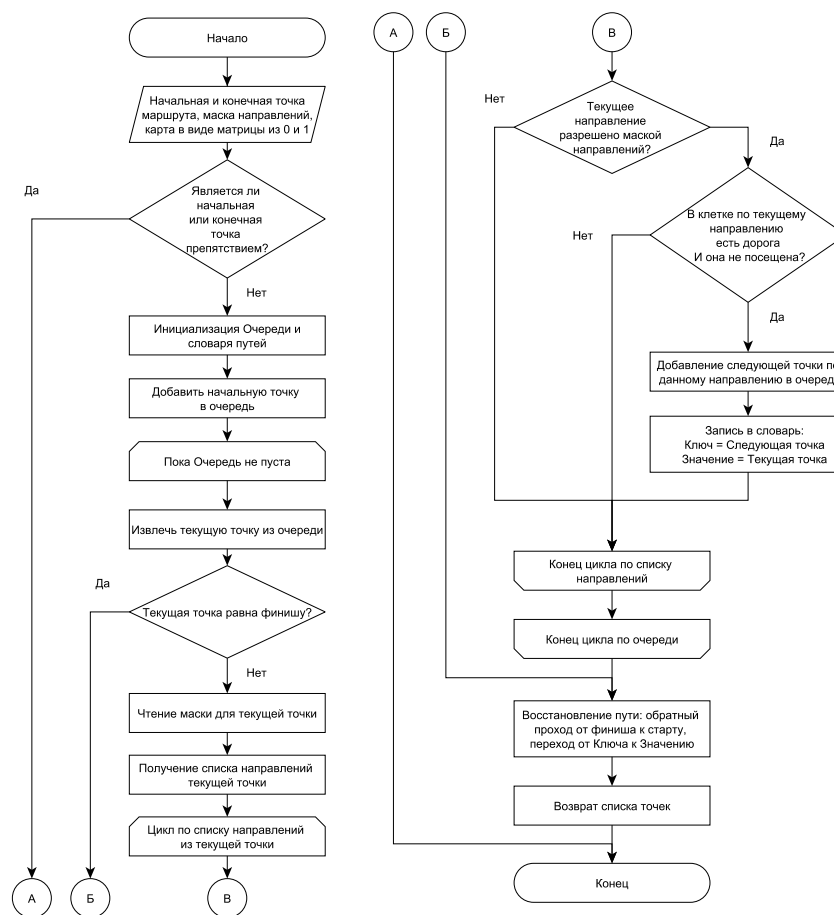


Рисунок 2.3 — Схема модифицированного волнового алгоритма

2.10 Вывод

В разделе разработаны требования к ПО, определены ключевые структуры данных для хранения сцены. Детально описан конвейер визуализации, включающий матричные преобразования, алгоритм построочного сканирования с Z-буфером, метод карт теней и модель освещения Ламберта. Также представлена логика организации движения на основе волнового алгоритма.

3 Технологическая часть

В данной части обосновывается выбор средств реализации, приводится модульная структура разработанного программного обеспечения и описывается программная реализация ключевых алгоритмов визуализации, моделирования освещения и управления сценой.

3.1 Выбор средств реализации

Для разработки программного комплекса был выбран язык программирования **Python** версии 3.13 [11]. Выбор обусловлен следующими соображениями:

- 1) Наличие библиотеки **NumPy** [12], обеспечивающей векторизацию матричных операций. Поскольку рендеринг выполняется на центральном процессоре, использование низкоуровневых оптимизаций NumPy, реализованных на языке C, необходимо для повышения производительности вычислений.
- 2) Кроссплатформенность, позволяющая исполнять приложение в операционных системах Windows, Linux и macOS без перекомпиляции исходного кода.
- 3) Высокоуровневый синтаксис и динамическая типизация, упрощающие реализацию алгоритмической части.

В качестве библиотеки графического интерфейса пользователя (GUI) используется **PyQt5** [13]. Это набор привязок к фреймворку Qt, реализующий механизм сигналов и слотов [14]. Библиотека предоставляет набор элементов управления и средства работы с растровой графикой (классы QImage и QPixmap), используемые для вывода буфера кадра.

Разработка выполнялась в интегрированной среде **PyCharm**, содержащей инструменты отладки, рефакторинга и профилирования кода. Для сборки приложения в исполняемый файл применялся инструмент PyInstaller [15].

3.2 Структура программы

Программное обеспечение спроектировано с использованием модульной архитектуры, что упрощает поддержку и расширение функционала. Архитек-

тура приложения и зависимости между модулями визуально представлены на диаграмме пакетов (рисунок 3.1).

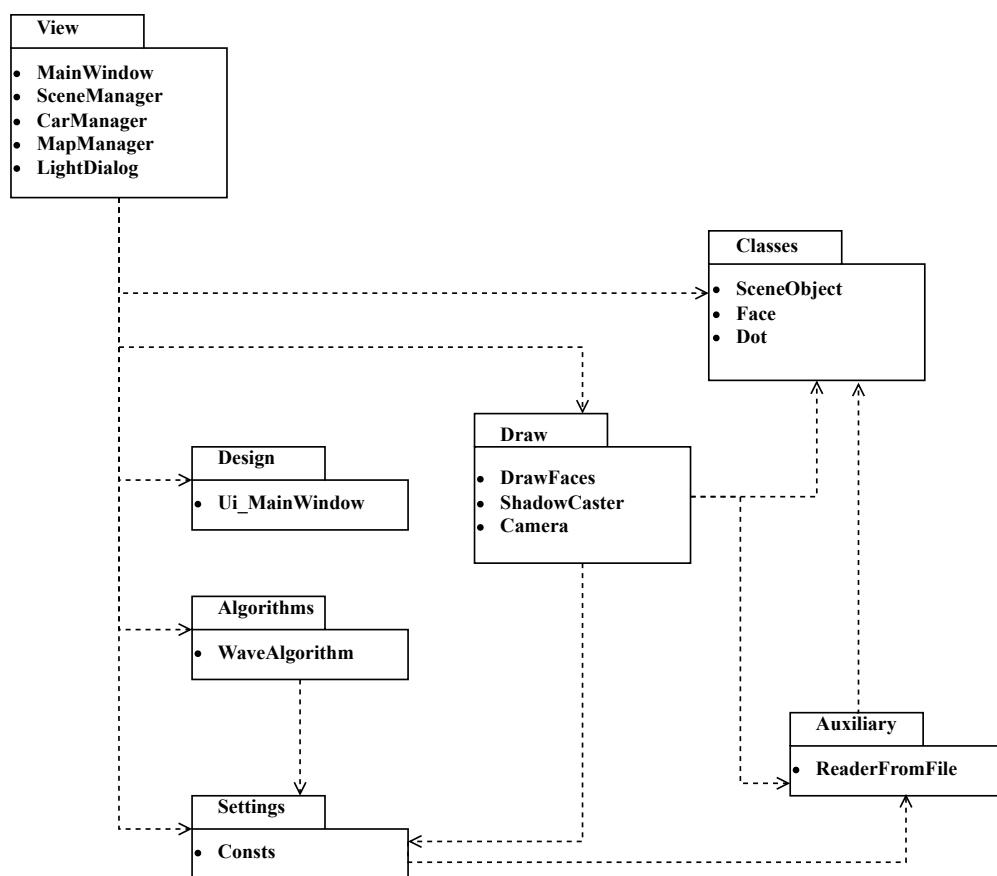


Рисунок 3.1 — Диаграмма пакетов программного средства

Основные модули и их назначение:

- **main.py** — точка входа в приложение, инициализация главного окна.
- **settings/consts.py** — файл конфигурации, содержащий глобальные константы (разрешение рендеринга, параметры камеры, пути к ресурсам).
- **view/** — пакет, отвечающий за пользовательский интерфейс и логику взаимодействия:
 - **main_window.py** — класс главного окна, обработка событий таймера и ввода.
 - **scene_manager.py** — менеджер сцены, связывающий математическую модель с виджетом вывода.
 - **light_dialog.py** — диалоговое окно настройки источников

света.

- **draw/** — пакет, содержащий реализацию графического конвейера:
 - `draw_faces.py` — ядро рендеринга: алгоритмы растеризации, Z-буфер, расчет освещения.
 - `shadows.py` — класс для генерации карт теней.
 - `camera.py` — математическая модель камеры (матрицы вида и проекции).
- **algorithms/** — пакет вспомогательных алгоритмов (поиск пути).
- **classes/** — описания структур данных (`SceneObject`, `Dot`, `Face`).

3.3 Реализация алгоритмов визуализации

Одной из наиболее трудоемких задач является реализация алгоритма построения теней (Shadow Mapping). Для этого был разработан класс `ShadowCaster`, который рассчитывает матрицы вида и проекции для источника света и формирует карту глубины.

В листинге 3.1 представлен код класса, отвечающего за создание матриц трансформации для источника света и рендеринг карты теней.

Листинг 3.1 — Реализация класса `ShadowCaster` (файл `draw/shadows.py`)

```
import numpy as np
from settings.consts import SHADOW_MAP_RES, LIGHT_ORTHO_SIZE

class ShadowCaster:
    def __init__(self, light_config):
        raw_dir = np.array(light_config['dir'], dtype=float)
        norm = np.linalg.norm(raw_dir)
        if norm == 0:
            self.light_dir = np.array([0.0, 1.0, 0.0])
        else:
            self.light_dir = raw_dir / norm
        self.color = light_config['color']
        self.intensity = light_config['intensity']
        self.resolution = SHADOW_MAP_RES
        self.ortho_size = LIGHT_ORTHO_SIZE
        self.shadow_buffer = np.full((self.resolution, self.resolution), np.inf,
                                     dtype=np.float32)
        self.view_matrix = np.eye(4)
        self.proj_matrix = np.eye(4)
        self.vp_matrix = np.eye(4)
        self._calculate_matrices()
```

```

def _calculate_matrices(self):
    target = np.array([0.0, 0.0, 0.0])
    position = target + self.light_dir * 50.0
    up = np.array([0.0, 1.0, 0.0])
    if abs(np.dot(self.light_dir, up)) > 0.95:
        up = np.array([0.0, 0.0, 1.0])
    forward = target - position
    forward /= np.linalg.norm(forward)
    right = np.cross(forward, up)
    right /= np.linalg.norm(right)
    real_up = np.cross(right, forward)
    self.view_matrix = np.eye(4)
    self.view_matrix[0, :3] = right
    self.view_matrix[1, :3] = real_up
    self.view_matrix[2, :3] = -forward
    self.view_matrix[:3, 3] = -np.array([
        np.dot(right, position),
        np.dot(real_up, position),
        np.dot(-forward, position)
    ])
    r = self.ortho_size
    l = -self.ortho_size
    t = self.ortho_size
    b = -self.ortho_size
    n = 1.0
    f = 100.0
    self.proj_matrix = np.array([
        [2 / (r - l), 0, 0, -(r + l) / (r - l)],
        [0, 2 / (t - b), 0, -(t + b) / (t - b)],
        [0, 0, -2 / (f - n), -(f + n) / (f - n)],
        [0, 0, 0, 1]
    ])
    self.vp_matrix = self.proj_matrix @ self.view_matrix

def transform_poly_to_light_space(self, vertices):
    res_coords = []
    for v in vertices:
        vec = np.array([v.x, v.y, v.z, 1.0])
        clip = self.vp_matrix @ vec
        w = clip[3] if clip[3] != 0 else 1.0
        ndc_x = clip[0] / w
        ndc_y = clip[1] / w
        ndc_z = clip[2] / w
        screen_x = (ndc_x + 1) * 0.5 * (self.resolution - 1)
        screen_y = (1 - ndc_y) * 0.5 * (self.resolution - 1)

```

```

depth = ndc_z * 0.5 + 0.5
res_coords.append((screen_x, screen_y, depth))
return res_coords

def render_shadow_map(self, faces):
    self.shadow_buffer.fill(np.inf)
    for face in faces:
        coords = self.transform_poly_to_light_space(face.vertices)
        xs = [c[0] for c in coords]
        ys = [c[1] for c in coords]
        if max(xs) < 0 or min(xs) >= self.resolution or max(ys) < 0 or min(ys) >=
            self.resolution:
            continue
        n_verts = len(coords)
        min_y = max(0, int(np.floor(min(ys))))
        max_y = min(self.resolution - 1, int(np.ceil(max(ys))))
        for y in range(min_y, max_y + 1):
            intersections = []
            for i in range(n_verts):
                v1 = coords[i]
                v2 = coords[(i + 1) % n_verts]
                if (v1[1] <= y < v2[1]) or (v2[1] <= y < v1[1]):
                    if abs(v2[1] - v1[1]) < 1e-9: continue
                    t = (y - v1[1]) / (v2[1] - v1[1])
                    x_int = v1[0] + t * (v2[0] - v1[0])
                    z_int = v1[2] + t * (v2[2] - v1[2])
                    intersections.append((x_int, z_int))
            intersections.sort(key=lambda p: p[0])
            for k in range(0, len(intersections) - 1, 2):
                left = intersections[k]
                right = intersections[k + 1]
                x_start = max(0, int(np.ceil(left[0])))
                x_end = min(self.resolution - 1, int(np.ceil(right[0])))
                if x_end < x_start: continue
                width = right[0] - left[0]
                if width < 1e-9: continue
                xx = np.arange(x_start, x_end + 1)
                t_span = (xx - left[0]) / width
                z_vals = left[1] + t_span * (right[1] - left[1])
                current_depths = self.shadow_buffer[y, x_start:x_end + 1]
                mask = z_vals < current_depths
                self.shadow_buffer[y, x_start:x_end + 1][mask] = z_vals[mask]

```

3.4 Реализация алгоритма отрисовки сцены

Основной алгоритм визуализации реализован в модуле `draw_faces.py`. Он выполняет сбор геометрических данных, расчет освещения и наложение текстур.

В листинге 3.2 приведена функция, которая реализует построчное сканирование полигона (Scanline) с использованием Z-буфера и проверкой карты теней для каждого пикселя. Для ускорения работы внутренние циклы векторизованы с использованием массивов NumPy.

Листинг 3.2 — Реализация алгоритма растеризации грани (файл `draw/draw_faces.py`)

```
def rasterize_face_with_shadows(face, zbuffer, img_array, cam,
                                shadow_casters, current_scale):
    if len(face.vertices) < 3: return
    screen_coords = []
    for v in face.vertices:
        sx, sy, sz = project_vertex(v, cam, current_scale)
        if not (np.isfinite(sx) and np.isfinite(sy) and np.isfinite(sz)): return
        screen_coords.append((sx, sy, sz))
    xs = np.array([p[0] for p in screen_coords])
    ys = np.array([p[1] for p in screen_coords])
    min_x, max_x = int(np.floor(np.min(xs))), int(np.ceil(np.max(xs)))
    min_y, max_y = int(np.floor(np.min(ys))), int(np.ceil(np.max(ys)))
    if max_x < 0 or min_x >= WIDTH_CANVAS or max_y < 0 or min_y >=
        HEIGHT_CANVAS: return
    min_y = max(0, min_y)
    max_y = min(HEIGHT_CANVAS - 1, max_y)
    color_key = tuple(face.color)
    is_textured = (face.uv is not None and color_key in TEXTURES and TEXTURES
        [color_key] is not None)
    texture_data = TEXTURES[color_key] if is_textured else None
    uv_coords = face.uv if is_textured else None
    texture_repeat = TEXTURE_REPEAT_MAP.get(color_key, DEFAULT_TEXTURE_REPEAT
        )
    normal = calculate_face_normal(face.vertices)
    casters_data = []
    for caster in shadow_casters:
        light_dir = caster.light_dir
        diff = abs(np.dot(normal, light_dir))
        s_coords = caster.transform_poly_to_light_space(face.vertices)
        casters_data.append({'caster': caster, 'diffuse': diff, 's_coords':
            s_coords})
    n_verts = len(face.vertices)
```

```

for y in range(min_y, max_y + 1):
    intersections = []
    for i in range(n_verts):
        v1 = screen_coords[i]
        v2 = screen_coords[(i + 1) % n_verts]
        y1, y2 = v1[1], v2[1]
        if (y1 <= y < y2) or (y2 <= y < y1):
            if abs(y2 - y1) < 1e-9: continue
            t = (y - y1) / (y2 - y1)
            x_int = v1[0] + t * (v2[0] - v1[0])
            z_int = v1[2] + t * (v2[2] - v1[2])
            sc_list = []
            for c_data in casters_data:
                s_poly = c_data['s_coords']
                s1 = s_poly[i]
                s2 = s_poly[(i + 1) % n_verts]
                s_int = (s1[0] + t * (s2[0] - s1[0]), s1[1] + t * (s2[1] - s1[1]), s1[2]
                    + t * (s2[2] - s1[2]))
                sc_list.append(s_int)
            uvc = (0,0)
            if is_textured:
                u1, v1_uv = uv_coords[i]
                u2, v2_uv = uv_coords[(i + 1) % n_verts]
                uvc = (u1 + t*(u2-u1), v1_uv + t*(v2_uv-v1_uv))
            intersections.append((x_int, z_int, sc_list, uvc))
    intersections.sort(key=lambda p: p[0])
    for k in range(0, len(intersections) - 1, 2):
        left = intersections[k]
        right = intersections[k+1]
        x_start = int(np.ceil(left[0]))
        x_end = int(np.ceil(right[0]))
        x_start = max(0, min(x_start, WIDTH_CANVAS - 1))
        x_end = max(0, min(x_end, WIDTH_CANVAS - 1))
        if x_end <= x_start: continue
        span_width = right[0] - left[0]
        if span_width < 1e-9: continue
        pixel_x = np.arange(x_start, x_end + 1, dtype=np.float32)
        t_span = (pixel_x - left[0]) / span_width
        z_left, z_right = left[1], right[1]
        z_vals = z_left + t_span * (z_right - z_left)
        current_z = zbuffer[y, x_start:x_end+1]
        visible_mask = z_vals < current_z
        if not np.any(visible_mask): continue
        indices = np.where(visible_mask)[0]
        full_indices = x_start + indices
        zbuffer[y, full_indices] = z_vals[indices]

```

```

t_vis = t_span[indices]
final_intensity = np.full(len(indices), AMBIENT_INTENSITY, dtype=np.
    float32)
for idx, c_data in enumerate(casters_data):
    caster = c_data['caster']
    diffuse = c_data['diffuse']
    sl = np.array(left[2][idx])
    sr = np.array(right[2][idx])
    s_vis = sl + t_vis[:, np.newaxis] * (sr - sl)
    map_x = s_vis[:, 0].astype(np.int32)
    map_y = s_vis[:, 1].astype(np.int32)
    v_d = s_vis[:, 2]
    in_map = (map_x >= 0) & (map_x < caster.resolution) & (map_y >= 0) & (
        map_y < caster.resolution) & (v_d >= 0) & (v_d <= 1.0)
    light_contrib = np.zeros(len(indices), dtype=np.float32)
    if np.any(in_map):
        valid_d = v_d[in_map]
        closest = caster.shadow_buffer[map_y[in_map], map_x[in_map]]
        lit_mask = valid_d <= (closest + SHADOW_BIAS)
        light_contrib[in_map] += np.where(lit_mask, 1.0, 0.0) * (diffuse * caster
            .intensity)
    final_intensity += light_contrib
    final_intensity = np.clip(final_intensity, 0.0, 1.0)
    if is_textured and texture_data:
        uv_l = np.array(left[3])
        uv_r = np.array(right[3])
        uv_vis = uv_l + t_vis[:, np.newaxis] * (uv_r - uv_l)
        tex_w = texture_data['width'] - 1
        tex_h = texture_data['height'] - 1
        tex_img = texture_data['array']
        u_fin = (uv_vis[:, 0] * texture_repeat) % 1.0
        v_fin = (uv_vis[:, 1] * texture_repeat) % 1.0
        tx = (u_fin * tex_w).astype(np.int32)
        ty = (v_fin * tex_h).astype(np.int32)
        base_colors = tex_img[ty, tx].astype(np.float32)
    else:
        base_colors = np.array(face.color, dtype=np.float32)
    res_colors = base_colors * final_intensity[:, np.newaxis]
    img_array[y, full_indices, 0] = res_colors[:, 2]
    img_array[y, full_indices, 1] = res_colors[:, 1]
    img_array[y, full_indices, 2] = res_colors[:, 0]

```

В листинге 3.3 приведена функция, объединяющая все этапы конвейера: инициализацию буферов, генерацию теней для всех источников и финальную отрисовку.

Листинг 3.3 — Управляющая функция отрисовки сцены (файл draw/draw_faces.py)

```
def draw_scene_with_objects(label_field, faces, cam, current_scale,
    objects=[]):
    all_faces = []
    all_faces.extend(faces)
    for obj in objects:
        all_faces.extend(obj.transformed_faces())
    auto_uv_unwrap(all_faces)
    casters = init_lighting_system()
    if casters:
        for caster in casters:
            caster.render_shadow_map(all_faces)
    img_array = np.zeros((HEIGHT_CANVAS, WIDTH_CANVAS, 4), dtype=np.uint8)
    img_array[:, :, 3] = 255
    zbuffer = np.full((HEIGHT_CANVAS, WIDTH_CANVAS), np.inf, dtype=np.float32)
    for face in all_faces:
        rasterize_face_with_shadows(face, zbuffer, img_array, cam, casters,
            current_scale)
    height, width, channels = img_array.shape
    bytes_per_line = channels * width
    image = QImage(img_array.data, width, height, bytes_per_line, QImage.
        Format_RGB32).copy()
    label_field.setPixmap(QPixmap.fromImage(image))
```

3.5 Управление сценой и взаимодействие с пользователем

Класс `SceneManager` отвечает за обработку пользовательского ввода (клавиатура, мышь) и обновление параметров сцены, таких как положение камеры и масштаб отображения. Его реализация приведена в листинге 3.4.

Листинг 3.4 — Менеджер сцены (файл view/scene_manager.py)

```
class SceneManager:
    def __init__(self, label_field):
        self.label_field = label_field
        self.dots, self.edges, self.faces = [], [], []
        self.objects = []
        self.pressed_keys = set()
        self.current_scale = SCALE
        self.camera = Camera(
            position=DEFAULT_CAMERA_POSITION,
            target=DEFAULT_CAMERA_LOOK_AT,
            up=DEFAULT_CAMERA_UP,
            fov=DEFAULT_CAMERA_FOV
```

```

)
self.init_canvas()

def init_canvas(self):
self.canvas = QImage(WIDTH_CANVAS, HEIGHT_CANVAS, QImage.Format_RGB32)
self.canvas.fill(Qt.black)
self.label_field.setPixmap(QPixmap.fromImage(self.canvas))

def load_scene_data(self):
reader_from_file(filename=FILENAME_MAP, dots=self.dots, edges=self.edges,
faces=self.faces)

def add_object(self, obj):
self.objects.append(obj)

def update_scene(self):
if Qt.Key_Q in self.pressed_keys:
self.current_scale += ZOOM_SPEED
if Qt.Key_E in self.pressed_keys:
self.current_scale -= ZOOM_SPEED
self.current_scale = max(SCALE, min(self.current_scale, MAX_SCALE))
update_action(self.label_field, self.faces, self.pressed_keys, self.
camera, self.current_scale, objects=self.objects)

def key_press_event(self, event):
if event.isAutoRepeat(): return
self.pressed_keys.add(event.key())

def key_release_event(self, event):
if event.isAutoRepeat(): return
if event.key() in self.pressed_keys:
self.pressed_keys.remove(event.key())

```

3.6 Описание интерфейса программы

Главное окно программы (рисунок 3.2) разделено на две функциональные области. Слева находится область визуализации, занимающая основную часть экрана. Здесь отображается трехмерная сцена города, автомобиль и элементы интерфейса, такие как счетчик кадров в секунду.

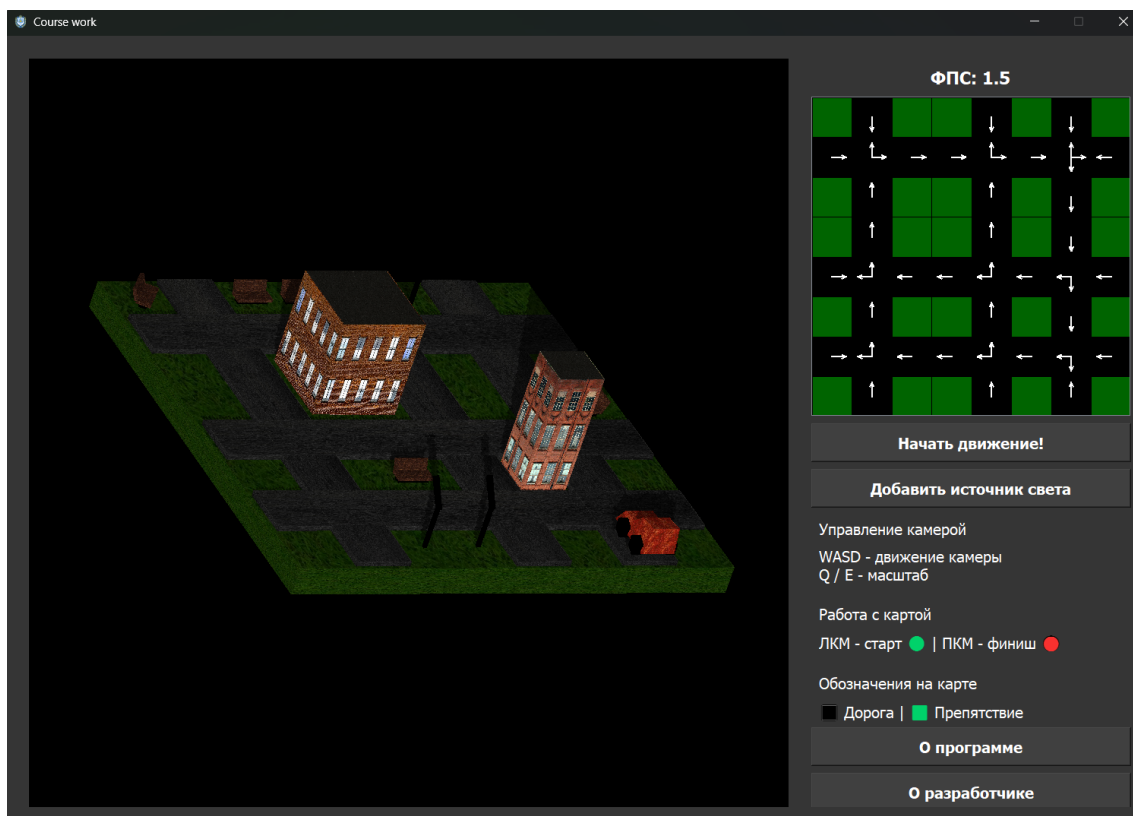


Рисунок 3.2 — Главное окно программы

Справа расположена панель управления. В её верхней части находится интерактивная карта (рисунок 3.3), которая визуализирует топологию дорожной сети. С помощью карты пользователь задает начальную (зеленый маркер) и конечную (красный маркер) точки маршрута, используя левую и правую кнопки мыши соответственно. Также на карте отображаются разрешенные направления движения в виде стрелок.



Рисунок 3.3 — Интерфейс выбора маршрута (мини-карта)

Для управления условиями освещения предусмотрено модальное окно (рисунок 3.4), вызываемое по кнопке «Добавить источник света». Оно позволяет задать вектор направления света и его интенсивность.

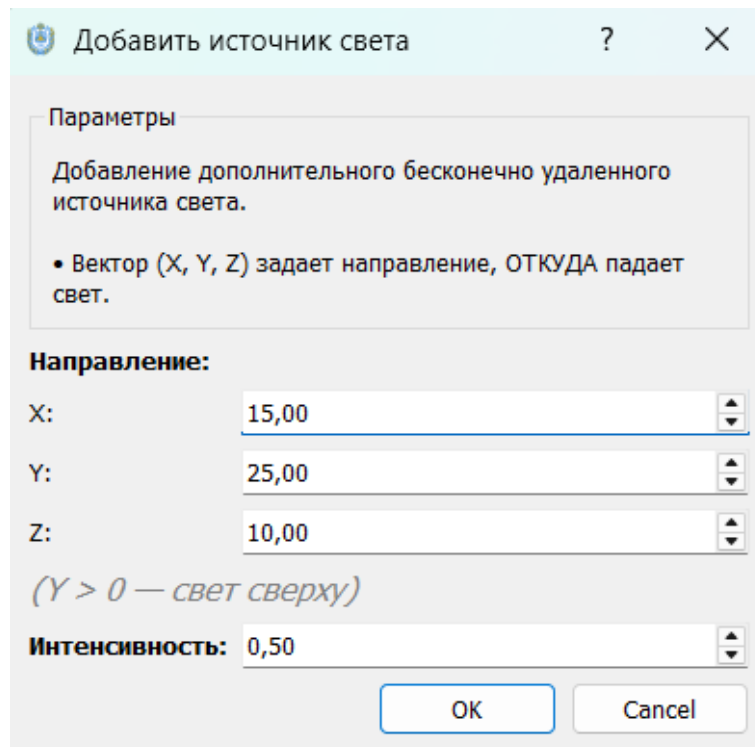


Рисунок 3.4 — Диалоговое окно настройки освещения

3.7 Выводы по технологической части

В данном разделе был обоснован выбор языка Python и библиотеки PyQt5 для реализации программного комплекса. Подробно описана структура проекта и реализация ключевых алгоритмов визуализации, включая построение сканирование, Z-буфер и Shadow Mapping. Приведены листинги кода основных модулей. Разработанное программное обеспечение обладает модульной структурой, что облегчает его дальнейшее сопровождение и развитие.

4 Исследовательская часть

В данном разделе приводится анализ производительности разработанного программного комплекса. Поскольку система реализует алгоритмы трёхмерной визуализации средствами центрального процессора без использования аппаратного ускорения видеокарты, критически важным является исследование временных затрат на генерацию кадра и выявление факторов, влияющих на быстродействие.

4.1 Цель и задачи исследования

Целью исследования является оценка зависимости времени генерации одного кадра изображения от коэффициента масштабирования сцены.

В ходе исследования решались следующие **задачи**:

- 1) внедрение в программный код средств профилирования для измерения чистого времени рендеринга.
- 2) проведение серии замеров при плавном изменении коэффициента масштабирования от начального значения до максимального.
- 3) анализ полученной зависимости и оценка эффективности алгоритмов отсечения невидимой геометрии.

4.2 Технические характеристики стенда

Экспериментальное исследование проводилось на ноутбуке со следующей аппаратной и программной конфигурацией:

- **Центральный процессор (CPU):** AMD Ryzen 7 8845HS, тактовая частота 3.8 ГГц;
- **Оперативная память (RAM):** 32 ГБ;
- **Операционная система:** Microsoft Windows 11 (64-bit);
- **Среда выполнения:** Интерпретатор Python 3.13.

Для обеспечения стабильности и точности замеров были выполнены следующие действия:

- 1) подключение ноутбука к источнику бесперебойного питания;
- 2) завершение всех остальных пользовательских приложений, не использующихся для замеров времени.

Замеры времени выполнялись с использованием системного таймера с точностью до одной десятой миллисекунды.

4.3 Методика проведения исследования

Для получения данных использовалась следующая методика:

- 1) камера устанавливалась в начальное положение (вид сверху); исходный коэффициент масштабирования (Scale) задавался равным увеличению изображения в **40 раз**;
- 2) производилось плавное увеличение коэффициента масштабирования от **40-кратного** до **150-кратного** увеличения, при этом фиксировалось время генерации каждого кадра;
- 3) для исключения случайных выбросов, замеры для каждого коэффициента масштабирования проводились **10 раз**, а полученные значения усреднялись.

4.4 Результаты исследования

В таблице 4.1 приведены полные результаты замеров времени отрисовки кадра при плавном изменении коэффициента масштабирования от 40-кратного до 150-кратного увеличения с шагом 2.0. Графическая интерпретация зависимости представлена на рисунке 4.1.

Таблица 4.1 — Зависимость времени генерации кадра от коэффициента масштабирования

Scale	Время (мс)	FPS	Scale	Время (мс)	FPS
40,0	646,8	1,55	96,0	1104,7	0,90
42,0	678,4	1,47	98,0	1084,6	0,92
44,0	707,2	1,41	100,0	1024,9	0,98
46,0	739,5	1,35	102,0	1020,7	0,98
48,0	772,9	1,29	104,0	1073,9	0,93
50,0	811,6	1,23	106,0	1115,4	0,90
52,0	855,3	1,17	108,0	1046,1	0,96
54,0	836,7	1,19	110,0	1007,8	0,99
56,0	948,2	1,05	112,0	1021,3	0,98
58,0	1062,4	0,94	114,0	1145,4	0,87
60,0	1014,6	0,99	116,0	1020,8	0,98
62,0	1051,8	0,95	118,0	1048,3	0,95
64,0	1056,4	0,95	120,0	1012,6	0,99
66,0	1132,7	0,88	122,0	991,7	1,01
68,0	1058,9	0,94	124,0	1055,8	0,95
70,0	1131,5	0,88	126,0	1054,2	0,95
72,0	1139,8	0,88	128,0	978,3	1,02
74,0	1246,3	0,80	130,0	913,8	1,09
76,0	1260,7	0,79	132,0	947,5	1,06
78,0	1218,4	0,82	134,0	993,2	1,01
80,0	1253,6	0,80	136,0	971,8	1,03
82,0	1272,9	0,79	138,0	966,5	1,03
84,0	1208,7	0,83	140,0	957,3	1,04
86,0	1179,3	0,85	142,0	947,2	1,05
88,0	1154,8	0,87	144,0	917,5	1,09
90,0	1142,3	0,88	146,0	887,9	1,12
92,0	1146,7	0,87	148,0	937,4	1,07
94,0	1078,4	0,93	150,0	971,1	1,03



Рисунок 4.1 — График зависимости времени генерации кадра от коэффициента масштабирования

4.5 Анализ результатов

На основании полученных данных можно выделить характерные этапы изменения производительности, которые объясняются особенностями работы алгоритма построчного сканирования (Scanline Z-Buffer).

- 1) **Рост нагрузки (Scale 40.0 → 82.0).** В диапазоне масштабов от 40 до 82 наблюдается рост времени рендеринга с 646,8 мс до пикового значения 1272,9 мс. Это связано с тем, что при увеличении масштаба объекты сцены занимают всё большую площадь экрана. Поскольку алгоритм работает на CPU, его производительность ограничена скоростью заполнения пикселей.
- 2) **Пик нагрузки (Scale ≈ 82.0).** Максимальное время генерации кадра 1272,9 мс достигается при коэффициенте масштабирования 82.0. В этот момент сцена максимально заполняет видимую область экрана, но объекты еще практически не выходят за его границы. Процессор вынужден обрабатывать и закрашивать максимальное количество пикселей для всей геометрии сцены.
- 3) **Спад нагрузки и стабилизация (Scale > 82.0).** При дальнейшем уве-

личении масштаба (от 82 до 150) крупные объекты начинают выходить за границы кадра. В этот момент в алгоритме растеризации срабатывает механизм **отсечения (Clipping)**. Полигоны или их части, координаты которых находятся вне области видимости ($X < 0$ или $X > Width$), отбрасываются. Это снижает нагрузку на процессор, и время рендеринга снижается, стабилизируясь в диапазоне **900–1000 мс**.

4.6 Выводы по исследовательской части

Проведенный эксперимент позволил сделать выводы о работе разработанной систем.

- 1) Фактором, ограничивающим производительность, является количество пикселей сцены, которые в данный момент отображаются на экране. Приближение камеры увеличивает нагрузку до тех пор, пока объекты полностью помещаются в кадр.
- 2) Реализованный механизм отсечения и векторизация вычислений работают: при выходе объектов за границы кадра время рендеринга уменьшается, несмотря на то, что математически объекты становятся крупнее.
- 3) Система демонстрирует стабильную работу с FPS около 0.8–1.5 на сложной сцене с тенями в высоком разрешении (1200×1200) на центральном процессоре.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была успешно достигнута поставленная цель: разработан программный комплекс для трёхмерного моделирования и визуализации движения автомобиля в условиях городской застройки.

В процессе решения поставленных задач были получены результаты.

- 1) **Проведен анализ** предметной области и алгоритмической базы компьютерной графики. Обоснован выбор алгоритма построчного сканирования с использованием Z-буфера как наиболее надежного метода удаления невидимых поверхностей для сцен произвольной сложности.
- 2) **Спроектирована и реализована архитектура** графического приложения, включающая модули управления сценой, математического ядра и подсистему рендеринга. Выбранный стек технологий (Python + NumPy) позволил реализовать векторизованные вычисления на центральном процессоре.
- 3) **Реализован графический конвейер**. В приложение внедрены следующие методы визуализации:
 - построение динамических теней методом карт теней, что обеспечивает корректное самозатенение объектов;
 - плоское закрашивание с использованием модели освещения Ламберта;
 - UV-развертка для наложения текстур для повышения визуальной детализации дорожного полотна и зданий.
- 4) **Разработан модуль навигации**. Реализован модифицированный волновой алгоритм поиска пути на ориентированном графе, который учитывает не только геометрию препятствий, но и правила дорожного движения (разрешенные направления поворотов), задаваемые специальной матрицей.
- 5) **Выполнено исследование** производительности системы. Установлено, что фактором, влияющим на время генерации кадра, является коэффициент масштабирования сцены.

Разработанное программное обеспечение представляет собой законченную систему, демонстрирующую работу фундаментальных алгоритмов ком-

пьютерной графики и поиска пути. Полученные результаты могут быть использованы в учебных целях для демонстрации принципов работы графических движков, а также как база для создания симуляторов транспортных потоков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Dosovitskiy A., Ros G., Codevilla F. et al. CARLA: An Open Urban Driving Simulator. [Электронный ресурс]. 2017. Дата обращения: 09.12.2025. – URL: <https://arxiv.org/abs/1711.03938>.
2. Роджерс Д. Алгоритмические основы машинной графики. СПб.: БХВ-Петербург, 1989. 512 с.
3. Шикин Е. В., Боресков А. В. Компьютерная графика. Полигональные модели. М.: ДИАЛОГ-МИФИ, 2005. 461 с.
4. Shadow Volume. [Электронный ресурс]. Дата обращения: 08.12.2025. – URL: <https://gamedev.ru/code/terms/ShadowVolume>.
5. Williams L. Casting curved shadows on curved surfaces // ACM SIGGRAPH Computer Graphics. 1978. P. 270–274.
6. LearnOpenGL: Shadow Mapping. [Электронный ресурс]. Дата обращения: 04.12.2025. – URL: <https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping>.
7. Algorithmica. Алгоритм Дейкстры. [Электронный ресурс]. Дата обращения: 09.12.2025. – URL: <https://ru.algorithmica.org/cs/shortest-paths/dijkstra/>.
8. Андрей Чураков Михаил и Якушев. Муравьиные алгоритмы. [Электронный ресурс]. 2011. Дата обращения: 09.12.2025. – URL: <https://masters.donntu.ru/2011/fknt/murzin/library/docs/3.htm>.
9. Волновой алгоритм (Алгоритм Ли). [Электронный ресурс]. Дата обращения: 08.12.2025. – URL: <https://suvitruf.ru/2012/05/13/1176/volnovojs-algoritma-algoritma-li/>.
10. Digital-Razor.ru. UV-mapping: советы и рекомендации. [Электронный ресурс]. Дата обращения: 09.12.2025. – URL: <https://digital-razor.ru/media/articles/workstation/uv-mapping-tips/>.

11. Python 3.13 Documentation. [Электронный ресурс]. Дата обращения: 04.12.2025. – URL: <https://docs.python.org/3.13/>.
12. NumPy v2.0 Manual. [Электронный ресурс]. Дата обращения: 04.12.2025. – URL: <https://numpy.org/doc/stable/>.
13. PyQt5 Reference Guide. [Электронный ресурс]. Дата обращения: 04.12.2025. – URL: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>.
14. Qt Documentation: Signals & Slots. [Электронный ресурс]. Дата обращения: 04.12.2025. – URL: <https://doc.qt.io/qt-6/signalsandslots.html>.
15. PyInstaller Manual. [Электронный ресурс]. Дата обращения: 04.12.2025. – URL: <https://pyinstaller.org/en/stable/>.

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе на тему «Визуализация движения автомобиля в трехмерной городской среде с использованием алгоритма поиска пути» состоит из N слайдов.